ntexup

A declarative, text-based document & presentation generator

Open Source

thevpc

Open Source Community

https://github.com/thevpc/ntexup

2026-09-13

Rationale

- Text-based, declarative, and intuitive syntax
- Readable by humans, writable with ease
- Designed for long-lived documents with effortless maintenance
- Unmatched control over rendering
- Seamless multi-file support
- Version-control friendly (**Git** & more)
- Integrates with LaTeX, UML, and beyond
- Uses **TSON** which is a derivative of JSON format but with more readability
- Declarative syntax : what you write is what you get
- Parameterizable : variables and conditions are processed for rendering
- Templatable : one can define his own components
- Themable : uses a **CSS** like styling
- Composable : you can combine multiple components to build reusable components
- Portable: works across platforms and environments with minimal setup

Hello World

Hello World

```
"Hello World"
```

- Hello World
- Hello World
- Hello World

```
"
    - Hello ##World##
    - Hello ###World###
    - Hello ####World####
"
```

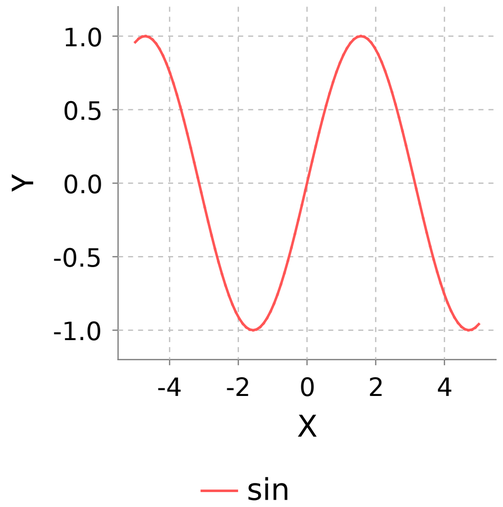
- Hello World
- Hello World
- Hello World

```
• ¶ Hello ##World##
• ¶ Hello ###World###
• ¶ Hello ####World####
```

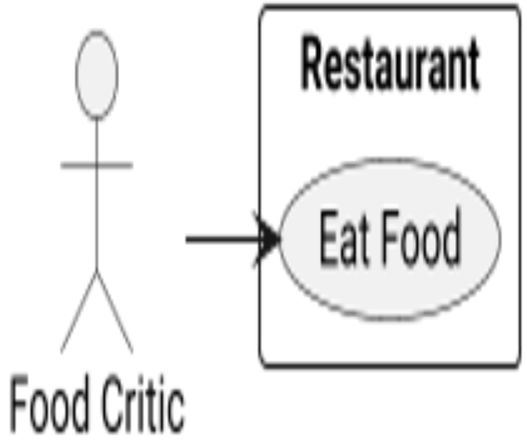
More Elaborate Example


```
// this is a comment
page{
    grid(2,2){
        group{ // supports shapes
            rectangle(at:center,size:(90),
                background-color:yellow,
                color:red, stroke:4
            )
            ul(origin:left, position:(20,50)){
                "Item 1" "Item 2"
            }
        }
        plot2d(x:dtimes(-5,5,100)){
            curve{ // supports plot
                f(x):sin(x)
                title:'sin'
            }
        }
        // supports LATEX
        eq("f_1(x)=\int_x \sin(x^2)",font-size:4%P)
        uml(// supports PLANTUML
            """
                left to right direction
                actor "Food Critic" as fc
                rectangle Restaurant {
                  usecase "Eat Food" as UC1
                }
                fc --> UC1
            """
        )
    }
}
```

- Item 1
- Item 2



$$f_1(x) = \int_x \sin(x^2)$$



Simple Text


```
plain("Hello World")
```

```
plain("Hello World",
      font-italic
)
```

```
plain(
    "
        Hello
        World
    "
    ,font-bold
)
```

```
plain(
    "
    Hello  World
    This    Is    Me
    "
    font-family: monospaced
)
```

NTF format

Hello World

```
ntf(
"""
##Hello## ### World ###
"""
)
```

color 1 color 2
color 3 color 4
italic hint

```
ntf(
"""
##:p1:color 1## ##:p2:color 2##
##:p3:color 3## ##:p10:color 4###
##:/:italic## ##:info hint##
"""
)
```

Latex Equations

$$X^2 = \sin(\pi x)$$

```
eq("X^2=\sin(\pi x)")
```

Latex math expressions are supported out of the box...

$$X^2 = \sin(\pi x)$$

```
eq("X²=\sin(πx)")
```

you can use superscripts and subscripts and custom symbols...

$$X^2 = \cos(\pi x)$$

```
"[[eq: X^2=\cos(x) ]]"
```

you can embed eq in a string...

Equation 1 (superscript) =
$X^2 = \sin(\pi x)$
Equation 2 =
$X^2 = \sin(\pi x)$
Equation 3 =
$X^2 = \sin(x)$

```
"""
##Equation 1 (superscript)## =
[[eq: X²=\sin(πx) ]]
###Equation 2### =
[[eq: X^2=\sin \left(\pi x \right)]]
####Equation 3#### =
[[eq: X^2=\sin(x) ]]
"""
```

you can embed eq in a string...

Source Code


```
public static class
MyClass{
    int value = 10;
    int add(int b){
        value++;
    }
}
```

```
source(java
"""
    public static class
    MyClass{
        int value = 10;
        int add(int b){
            value++;
        }
    }
"""
)
```

```
Select *
From Tab
Where 1=1
```

```
source(sql
"""
    Select *
    From Tab
    Where 1=1
"""
)
```

```
<a value="text">
  <b value="text"></b>
</a>
```

```
source(xml
"""
    <a value="text">
      <b value="text"></b>
    </a>
"""
)
```

Supported Languages

```
text, java,
c#, c++
xml,html,json
bash,fish,cmd
sql,
hd, ntf,hadra,
tson,ntexup
```

Rich Text

Hello World

Hello World

Hello World

Hello World

```
"Hello World"
text("Hello World")
text("Hello World",
    font-italic)
text("Hello World",
    font-bold)
```

Bold X
__Italic Y__
#Title 1#
Title 2
Title 3

```
"""
**Bold X**
__Italic Y__
#Title 1#
##Title 2##
###Title 3###
"""
```

hello world

```
"""
[[ntf: ##:p1: hello##]] world
"""
```

#Equation 1 =
$$X^2 = \sin(\pi x)$$
Equation 2 =
$$X^2 = \sin\left(\pi x\right)$$
Equation 3 =
$$X^2 = \sin(x)$$

```
"""
#Equation 1# =
[[eq: X²=\sin(πx) ]]
##Equation 2## =
[[eq: X^2=\sin\left(\pi x\right)]]
###Equation 3### =
[[eq: X^2=\sin(x)]]
"""
```

Text Wrap & Alignment

Left aligned: wraps cleanly at the specified width boundaries.

Right aligned: lines end smoothly at the right boundary.

Centered: each wrapped line is centered within the box.

```
grid(1, 3){
    text("Left aligned...",
         text-wrap: wrap,
         text-align: left)
    text("Right aligned...",
         text-wrap: wrap,
         text-align: right)
    text("Centered...",
         text-wrap: wrap,
         text-align: center)
}
```

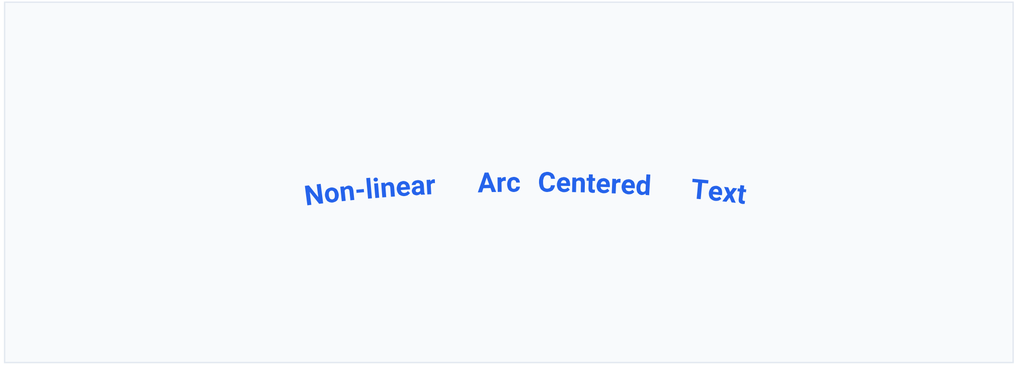
Justified text layout distributes available space evenly across all words on each line, creating crisp vertical margins on both sides while keeping the final paragraph line naturally left-aligned.

```
text(
    """
    Justified text layout
    distributes available space
    evenly across all words
    on each line, creating crisp
    margins on both sides...
    """,
    text-wrap: wrap,
    text-align: justify
)
```

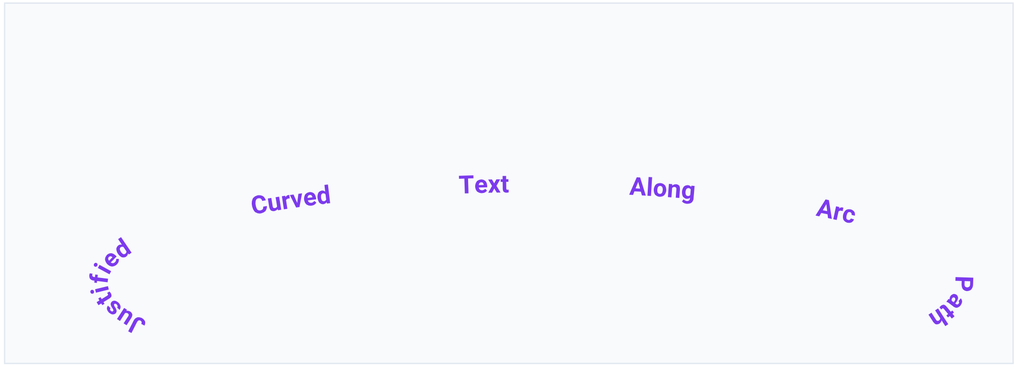
Melee text supports mixed plain words, **bold text**, and inline LaTeX formulas like $E = mc^2$ or $a^2 + b^2 = c^2$ with smooth wrapping and justification.

```
text(
    """
    Melee text supports mixed plain
    words, **bold text**, and inline
    LaTeX formulas like [[eq: E=mc^2 ]]
    or [[eq: a^2+b^2=c^2 ]]
    with smooth wrapping and
    justification.
    """,
    text-wrap: wrap,
    text-align: justify
)
```

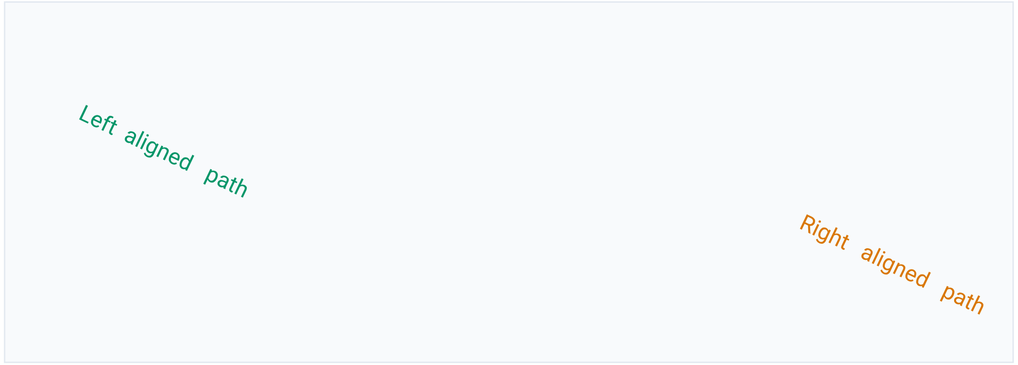
Text along Path




```
text(
    "Non-linear Arc Centered Text",
    text-path: {
        arc(cx: 50, cy: 75,
            rx: 42, ry: 25,
            from: -25, to: 205)
    },
    text-align: center,
    font-bold
)
```




```
text(
    "Justified Curved Text...",
    text-path: {
        arc(cx: 50, cy: 75,
            rx: 42, ry: 25,
            from: -25, to: 205)
    },
    text-align: justify,
    font-bold
)
```




```
text(
    "Left aligned path",
    text-path: { (5, 30), (35, 70), (65, 30), (95, 70)},
    text-align: left
)
text(
    "Right aligned path",
    text-path: {(5, 45), (35, 85),(65, 45), (95, 85)},
    text-align: right
)
```

Bullets

- Hello World
- Hello World
- Hello World

```
ul{
    ¶ Hello ##World##
    ¶ Hello ###World###
    ¶ Hello ####World####
}
```

- Hello World
  - Sub 1
  - Sub 2
- Hello World
- Hello World

```
ul{
    ¶ Hello ##World##
    ul{
        ¶ Sub 1
        ¶ Sub 2
    }
    ¶ Hello ###World###
    ¶ Hello ####World####
}
```

I. Hello World
   1. Sub 1
   2. Sub 2
II. Hello World
III. Hello World

```
ol{
    ¶ Hello ##World##
    ol{
        ¶ Sub 1
        ¶ Sub 2
    }
    ¶ Hello ###World###
    ¶ Hello ####World####
}
```

Bullets Alignment

bullet-align: top (for multi-line text):

- First item with a long multi-line description that wraps onto several lines cleanly. The bullet marker stays aligned with the first line.
- Second item also spanning multiple lines cleanly with top-aligned bullet marker.

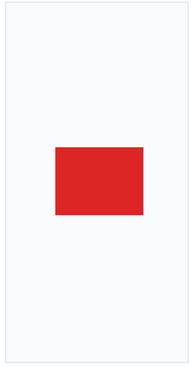
```
ul(bullet-align: top){
    text("First item multi-line...",
         text-wrap: wrap)
    text("Second item multi-line...",
         text-wrap: wrap)
}
```

bullet-align: center (for large objects):

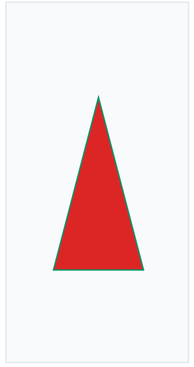
- 
- 

```
ul(bullet-align: center){
    rectangle(size: (32%P, 3.5%P),
              background-color: documentColors[4],
              color: documentColors[0])
    rectangle(size: (32%P, 3.5%P),
              background-color: documentColors[4],
              color: documentColors[1])
}
```

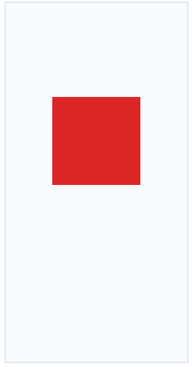
Shapes



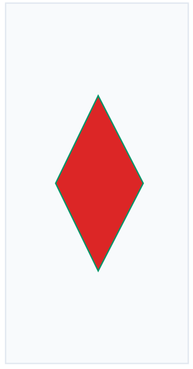
```
rectangle()
```



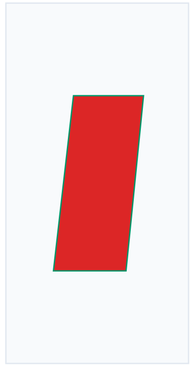
```
triangle()
```



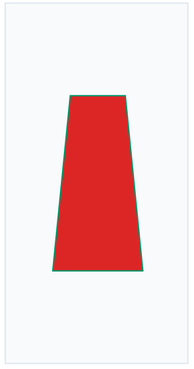
```
square()
```



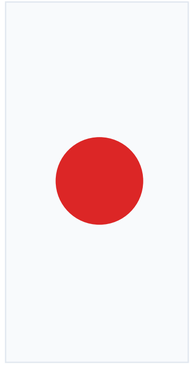
```
rhombus()
diamond()
```



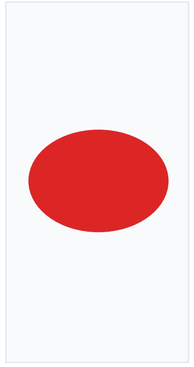
```
parallelogram()
```



```
trapezoid()
```

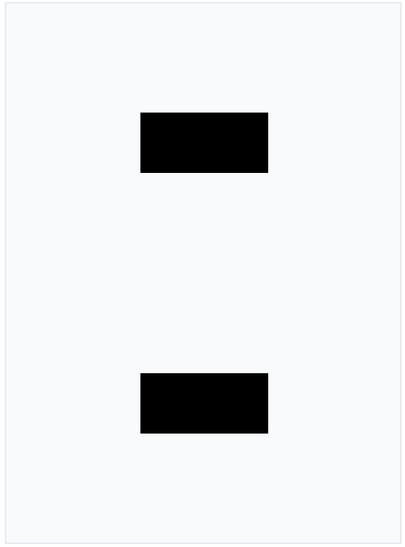


```
circle()
```

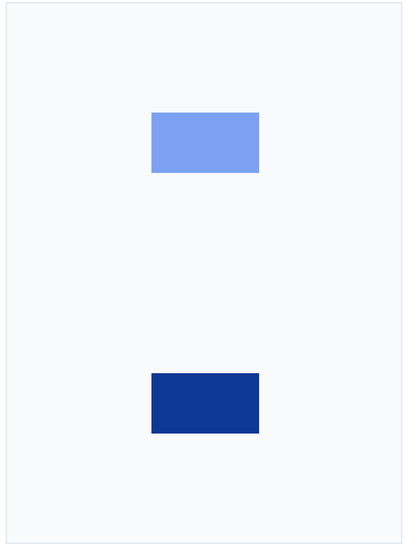


```
ellipse(
    size:(80,30)
)
```

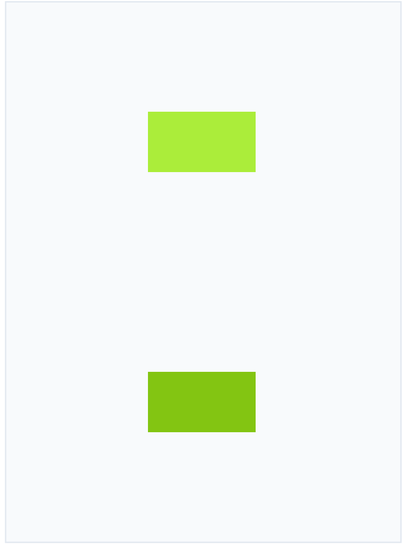
Color Functions & Gradients



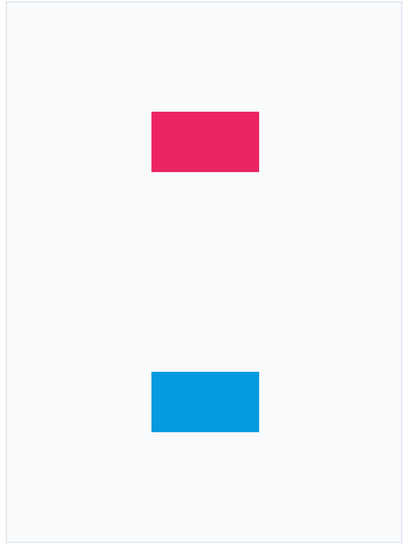
```
rectangle(size:(35,25),
  background-color:
    linearGradientColor(
      documentColors[0],
      documentColors[2]
    )
)
rectangle(size:(35,25),
  background-color:
    linearGradientColor(
      documentColors[1],
      documentColors[4],
      45
    )
)
```



```
rectangle(background-color:
  lighterColor(
    documentColors[0],
    0.4
  )
)
rectangle(background-color:
  darkerColor(
    documentColors[0],
    0.4
  )
)
```



```
rectangle(background-color:
  complementaryColor(
    documentColors[1]
  )
)
rectangle(background-color:
  invertColor(
    documentColors[1]
  )
)
```



```
rectangle(background-color:
  rotateHueColor(
    documentColors[0],
    120
  )
)
rectangle(background-color:
  sinebowColor(0.7)
)
```

Sizes



```
rectangle()
```



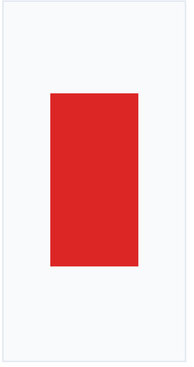
```
rectangle(
  size:(100,100)
)
```



```
rectangle(
    size:100
)
```



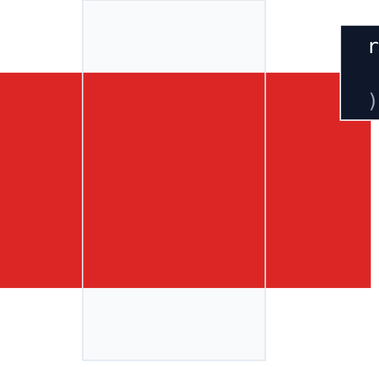
```
rectangle(
    size:80
)
```



```
rectangle(
    size:50
)
```

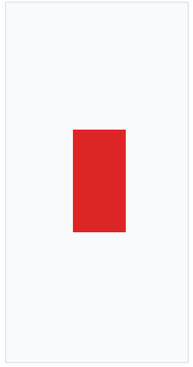


```
rectangle(
  size:(40,80)
)
```

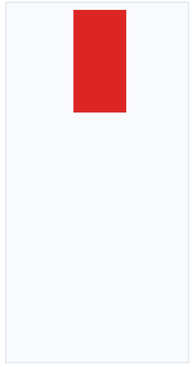


```
rectangle(
  size:15%P
)
```

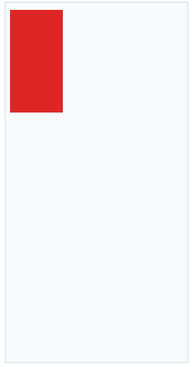
Positions



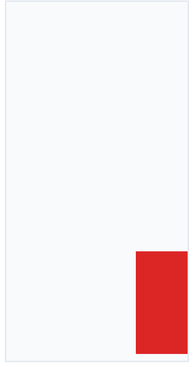
```
rectangle(
  at:center
)
```



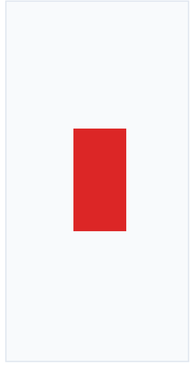
```
rectangle(
  at:top
)
```



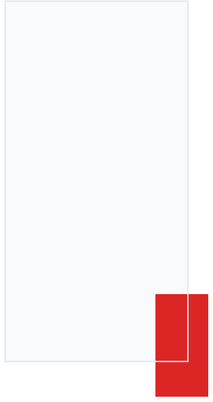
```
rectangle(
  at:top-left
)
```



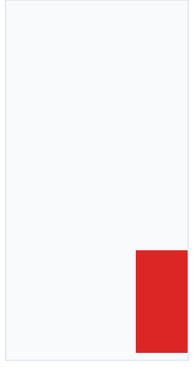
```
rectangle(
  at:bottom-right
)
```



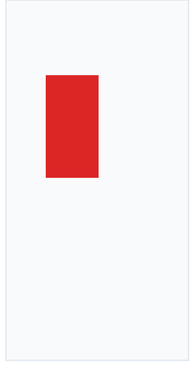
```
rectangle(
  origin:
    (50,50)
  position:
    (50,50)
)
```



```
rectangle(
  origin:
    (60,60)
  position:
    (100,100)
)
```

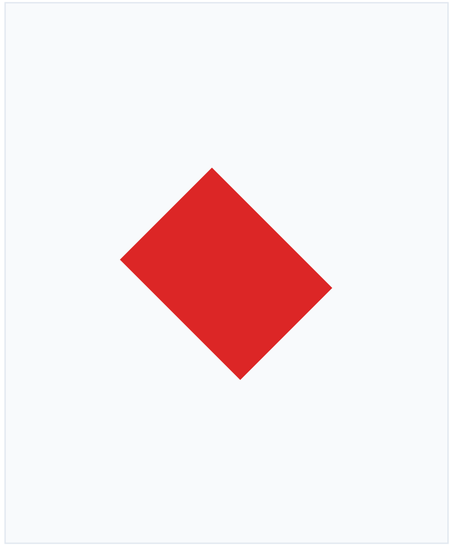


```
rectangle(
  origin:
    (100,100)
  position:
    (100,100)
)
```

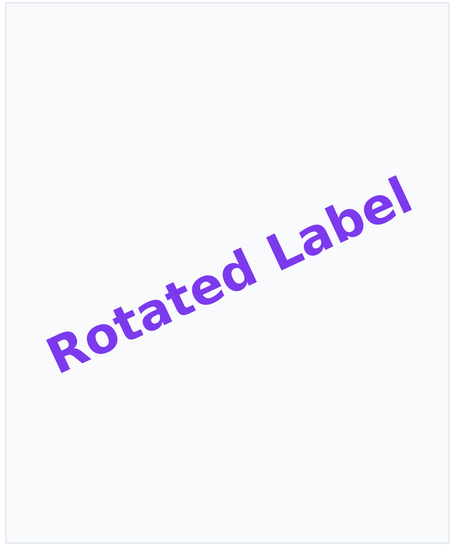


```
rectangle(
  origin:
    (100,100)
  position:
    (50,50)
)
```

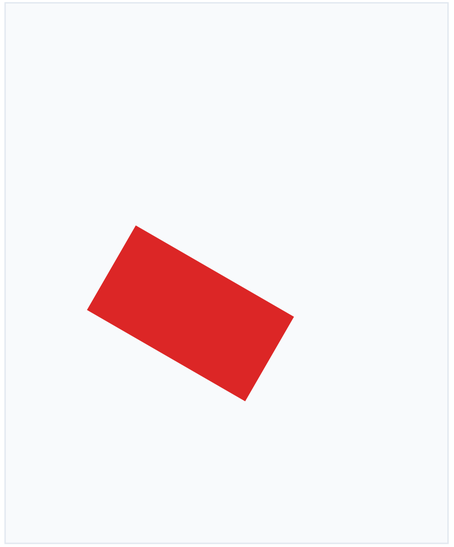
Rotation



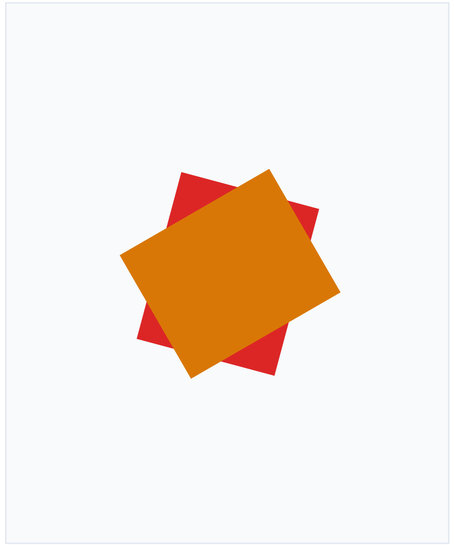
```
rectangle(
  at:center,
  size:(40,25),
  rotate:45
)
```




```
text(
  "Rotated Label",
  rotate:-25
)
```

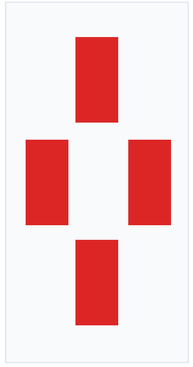


```
rectangle(
  at:center,
  size:(45,20),
  // pivot (angle, x, y):
  rotate:(30, 0, 0)
)
```

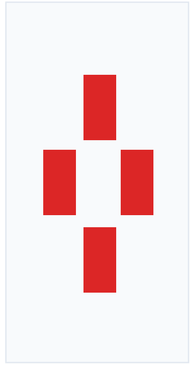


```
rectangle(size:(35,35), rotate:15)
rectangle(size:(35,35), rotate:60)
```

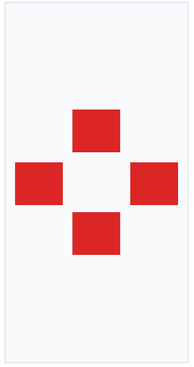
Margins



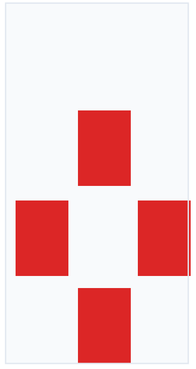
```
rectangle(
  margin:10
)
```



```
rectangle(
  margin:20
)
```



```
rectangle(
  margin:(5,30)
)
```



```
rectangle(
  margin:
    (5,30,0,0)
)
```

Strokes & Shadows



```
rectangle(
  size:(60,25),
  background-color:documentColors[4],
  draw-contour,
  line-color:documentColors[0],
  stroke:dashed
)
```



```
rectangle(
  size:(60,25),
  draw-contour,
  line-color:documentColors[1],
  stroke:basic(width:2,
               dash:[8,4],
               cap:round)
)
```

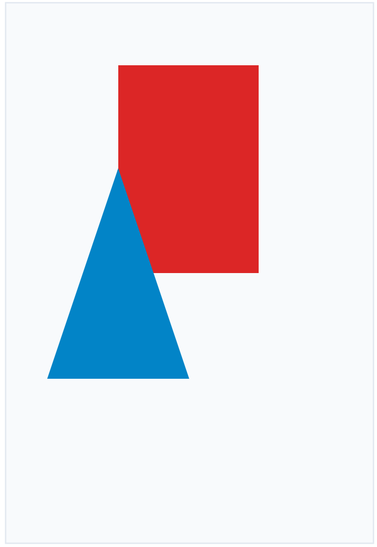


```
rectangle(
  size:(55,22),
  background-color:documentColors[0],
  shadow:{
    distance:(4,4),
    radius:6,
    color:gray
  }
)
```

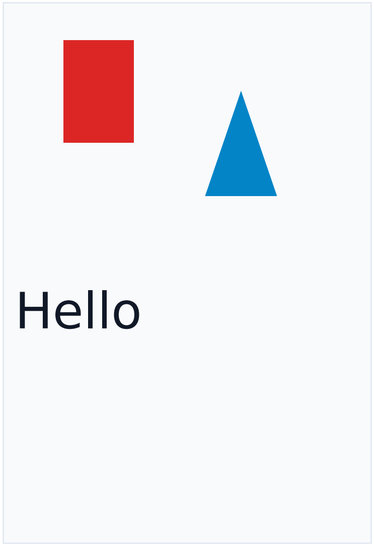



```
text(
  "Elevated Text",
  font-bold,
  color:documentColors[1],
  shadow:{
    distance:(3,3),
    radius:4,
    color:gray
  }
)
```

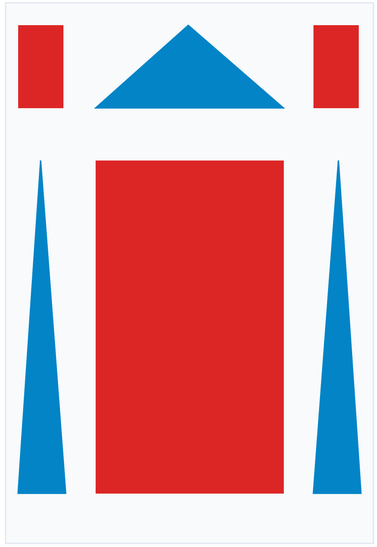
Layout



```
group{
  rectangle(at:top,
      size:(50,50),
      background-color:
        documentColors[4]
  )
  triangle(at:left,
      size:(50,50),
      background-color:
        documentColors[5]
  )
}
```

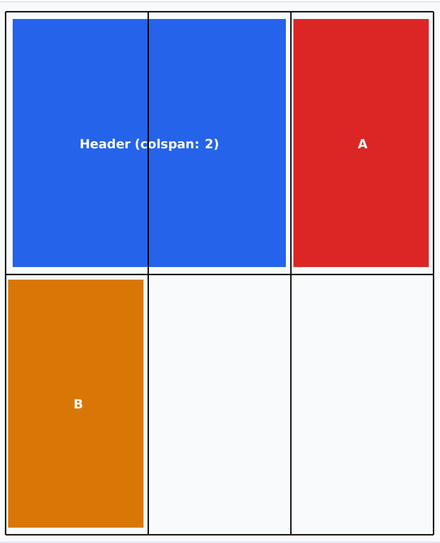



```
grid((2,2)){
  rectangle(at:top,
    size:(50,50),
    background-color:
      documentColors[4]
  )
  triangle(at:left,
    size:(50,50),
    background-color:
      documentColors[5]
  )
}
```

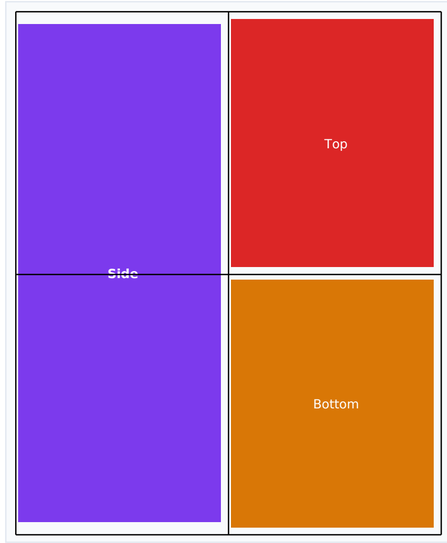


```
grid((3,2),
    columns-weight:[1,4],
    rows-weight:[1,4]){
        rectangle()
        triangle()
        rectangle()
        triangle()
        rectangle()
        triangle()
}
```

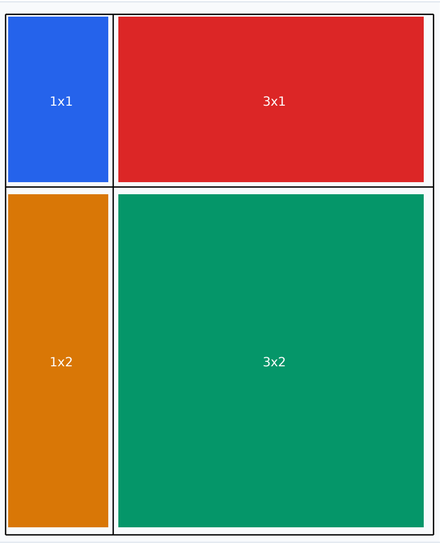
Grid Layouts & Spanning



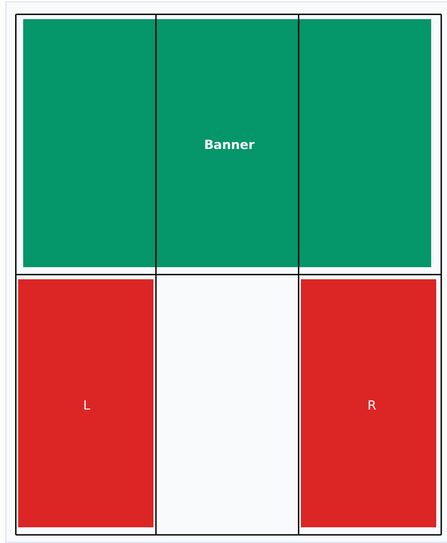

```
grid(3, 2, draw-grid){
  rectangle(colspan:2,
            background-color:documentColors[0]){
    text("Header (colspan: 2)",
         color:white)
  }
  rectangle(){ text("A") }
  rectangle(){ text("B") }
}
```




```
grid(2, 2, draw-grid){
  rectangle(rowspan:2,
            background-color:documentColors[1]){
    text("Side")
  }
  rectangle(){ text("Top") }
  rectangle(){ text("Bottom") }
}
```

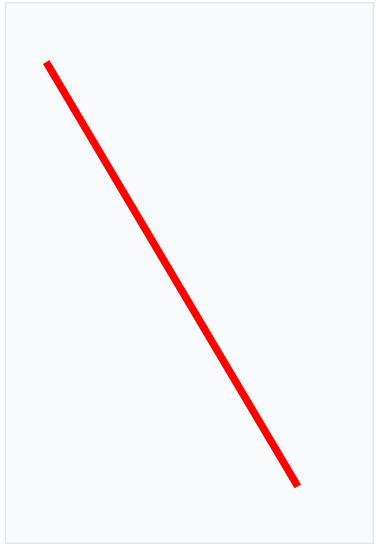



```
grid(2, 2, draw-grid,
     columns-weight:[1, 3],
     rows-weight:[1, 2]){
  rectangle(){ text("1x1") }
  rectangle(){ text("3x1") }
  rectangle(){ text("1x2") }
  rectangle(){ text("3x2") }
}
```




```
grid(3, 2, draw-grid){
  rectangle(colspan:3){
    text("Banner")
  }
  rectangle(){ text("L") }
  void() // empty spacer
  rectangle(){ text("R") }
}
```

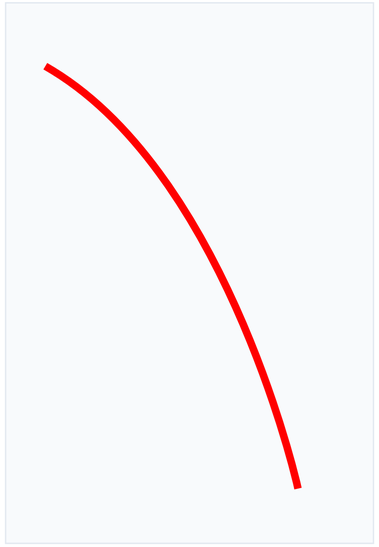
Lines



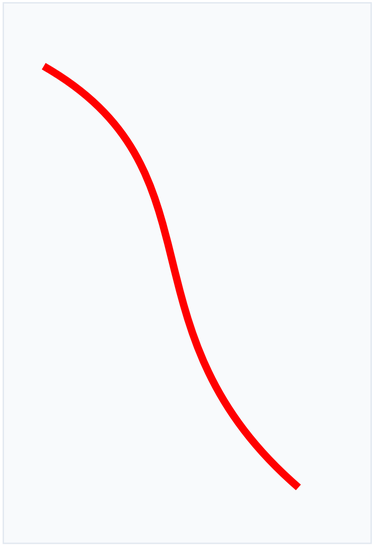
```
line(
  from:(0,0),
  to:(80,20)
)
```



```
arc(
    from:30,
    to:180
)
```

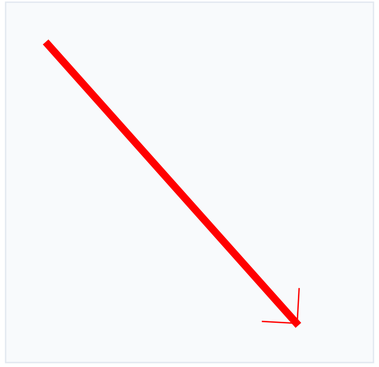


```
quad-curve(
    from:(10,10),
    ctrl:(60,30)
    to:(80,90),
)
```

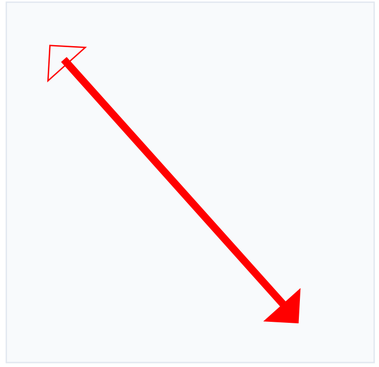


```
cubic-curve(
from:(10,10),
ctrl1:(60,30),
ctrl2:(30,60)
to:(80,90),
)
```

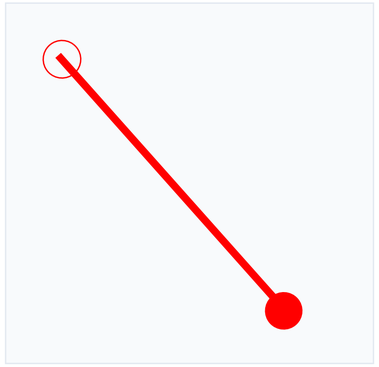
Arrows



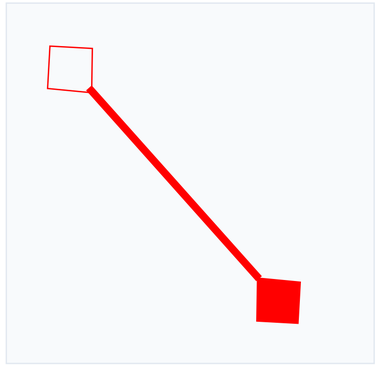
```
line(
    from:(10,10),
    to:(80,90)
    end-arrow:simple()
)
```



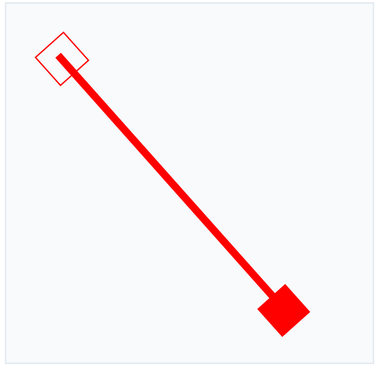
```
line(
    from:(10,10),
    to:(80,90)
    start-arrow:
      triangle()
    end-arrow:
      triangle-full()
)
```



```
line(
    from:(10,10),
    to:(80,90)
    start-arrow:
      circle()
    end-arrow:
      circle-full()
)
```

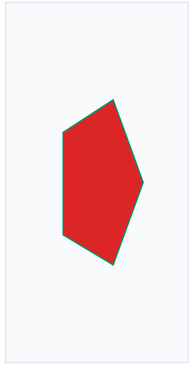


```
line(
    from:(10,10)
    to:(80,90)
    start-arrow:
      diamond()
    end-arrow:
      diamond-full()
)
```

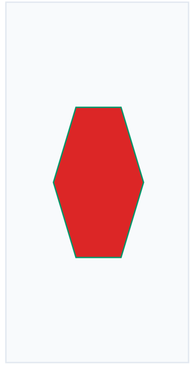


```
line(
    from:(10,10),
    to:(80,90)
    start-arrow:
      rectangle()
    end-arrow:
      rectangle-full()
)
```

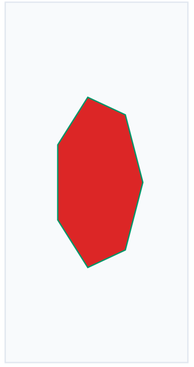
Polygons



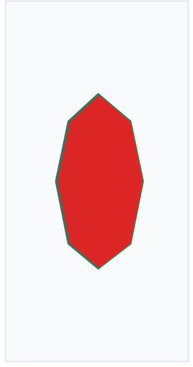
```
pentagon()
```



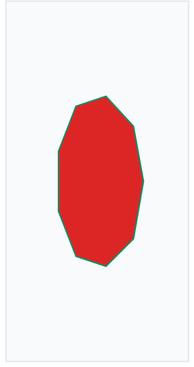
```
hexagon()
```



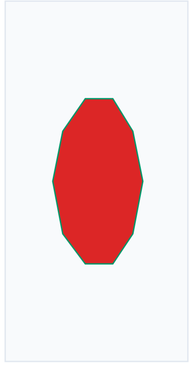
```
heptagon()
```



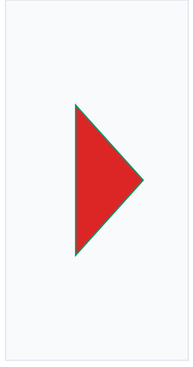
```
octagon()
```



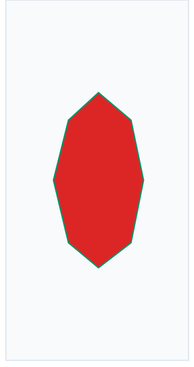
```
nonagon()
```



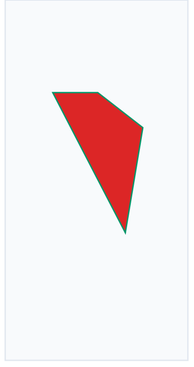
```
decagon()
```



```
polygon()
```



```
polygon(count:8)
```



```
polygon(points:[
(0,0),(50,0),
(100,80),(50,50)
])
```

Other Shapes



```
arrow(
  at: center,
  rotate:-45
)
```



```
cylinder(
  ellipse-height:20,
  segment-count:5
)
cylinder(
  ellipse-height:20,
  segment-count:3
)
cylinder(
  ellipse-height:20
)
```



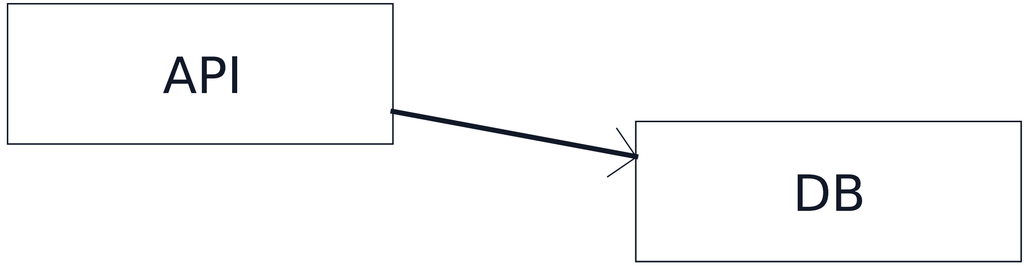
```
donut(
  inner-radius:50,
  start-angle:0,
  extent-angle:270
)
donut(inner-radius:30)
donut(inner-radius:80)
```



```
pie()
pie(
  start-angle:0,
  extent-angle:270
)
```

Connectors — endpoints

1. Automatic anchors



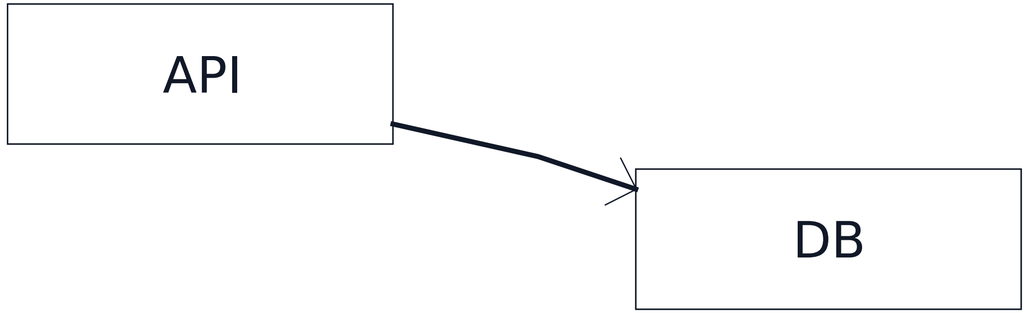

```
connector(
    "api", "db",
    end-arrow: simple()
)
```

2. Explicit sides



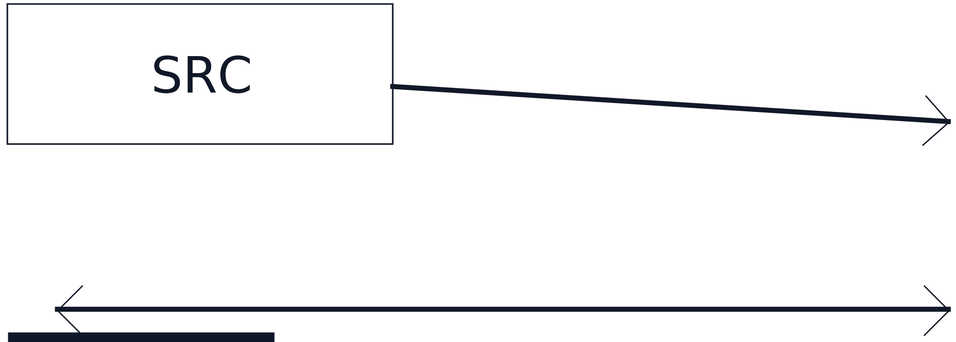

```
connector(
    from: "api@right", to: "db@left",
    end-arrow: triangle()
)
```

3. Via points




```
connector("api", "db",
    via: (24, 75),
    end-arrow: simple())
```

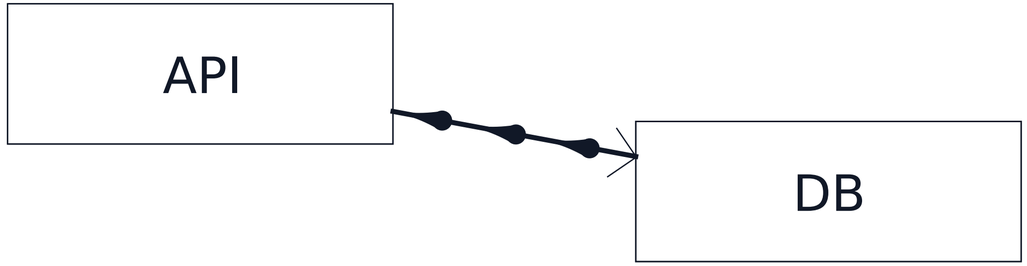
4. Free and mixed points




```
connector("src", (92, 72),
    end-arrow: simple())
connector((55, 88), (92, 88),
    end-arrow: simple())
```

Connectors — animated flow

1. Flow on automatic anchors



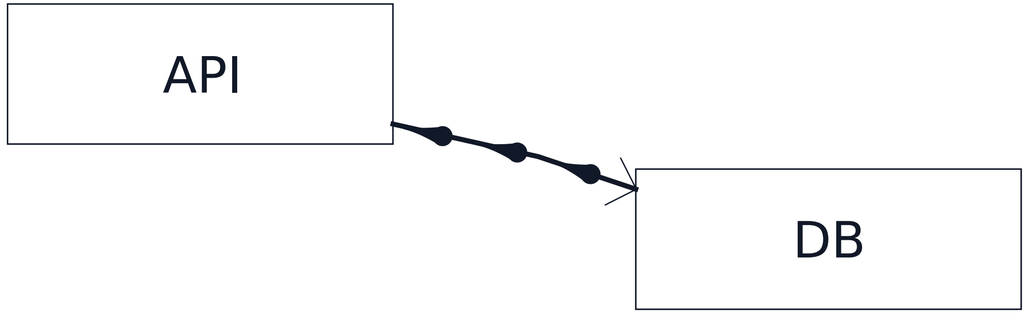

```
connector(
    "api", "db",
    flow: dots(count: 3)
)
```

2. Flow between sides



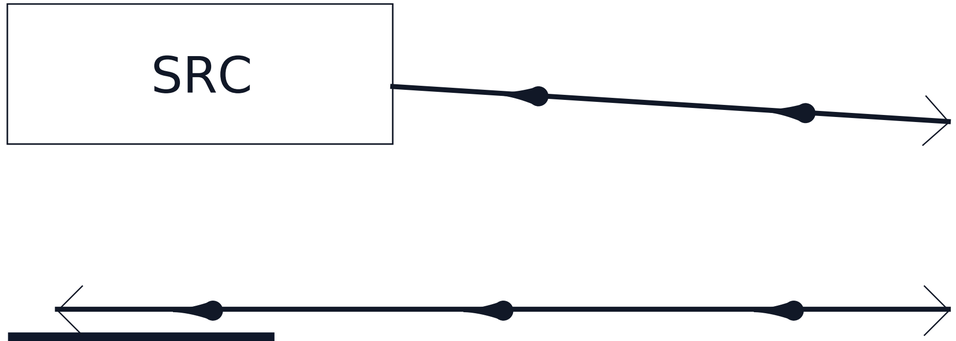

```
connector(
    from: "api@right", to: "db@left",
    flow: dots(count: 3)
)
```

3. Flow along via points




```
connector("api", "db",
    via: (24, 75),
    flow: dots(count: 3))
```

4. Flow on free and mixed points



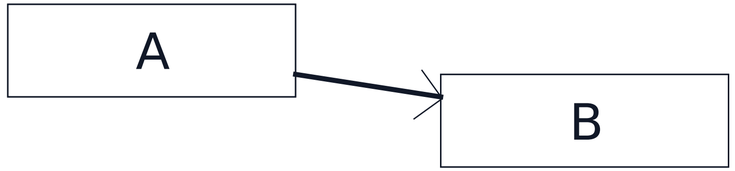

```
connector("src", (92, 72),
    flow: dots(count: 2))
connector((55, 88), (92, 88),
    flow: dots(count: 3))
```

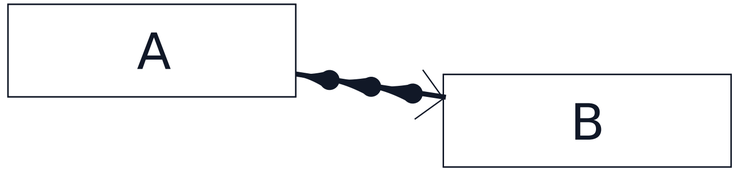
Connectors — static vs animated

static

animated

auto



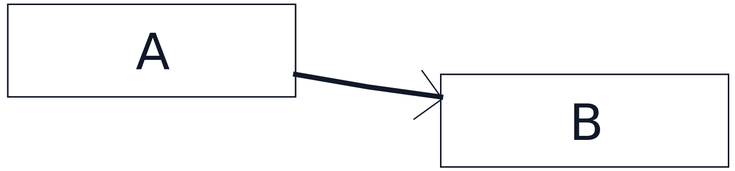


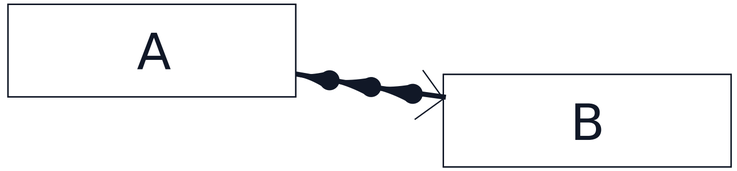
sides



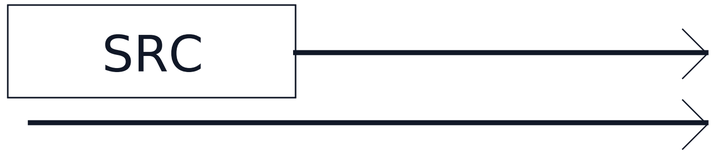


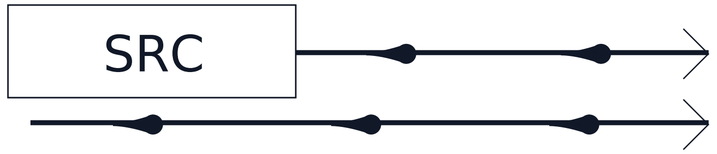
via





free





Static export uses elapsed 0; show mode animates the flow dots.

Lines and polylines — flow

line — static



```
line(
    from: (2, 10), to: (46, 10),
    end-arrow: triangle()
)
```

line — animated



```
line(
    from: (53, 10), to: (97, 10),
    flow: dots(count: 4)
)
```

polyline — static

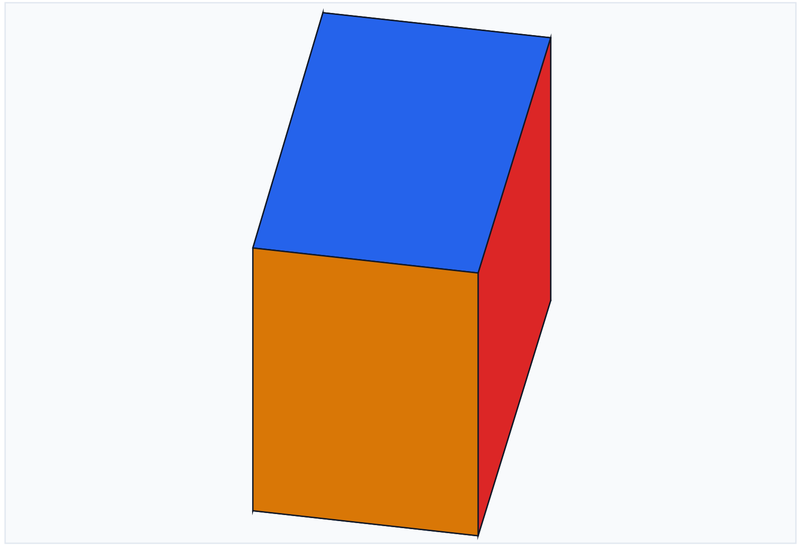
```
polyline(
    points: ((300, 700), (600, 600),
             (900, 800), (1200, 650)),
    end-arrow: triangle()
)
```

polyline — animated

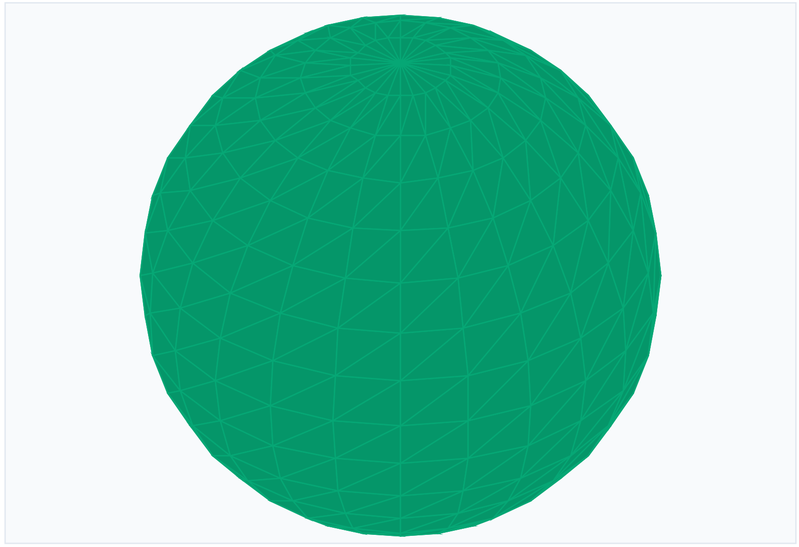
```
polyline(
    points: ((1400, 700), (1700, 600),
             (2000, 800), (2250, 650)),
    flow: dots(count: 4)
)
```

Static output shows evenly spaced flow dots; show mode animates them.

3D Primitives

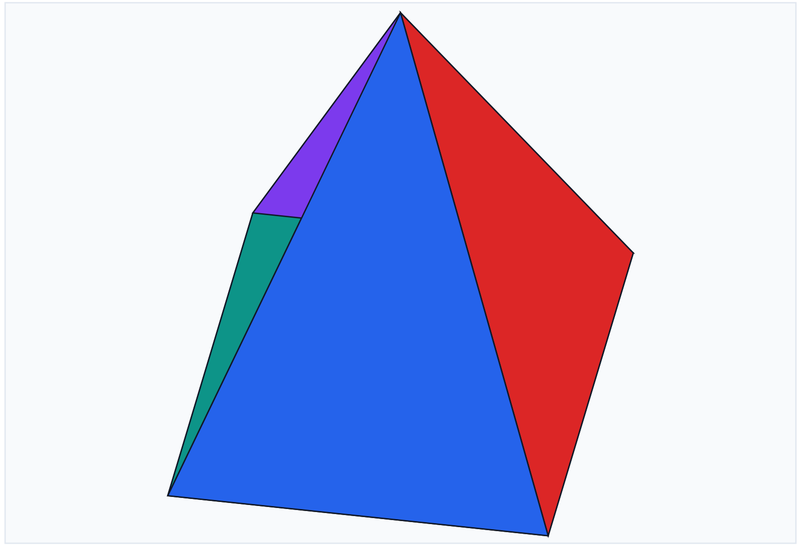


```
scene3d(real-position:(-22.5,-17.5,-12.5), real-size:(45,35,25)
  , transform:{
    rotate(to:(0,100,0), angle:35)
    scale:0.75
}){
  box(position:(-22.5,-17.5,-12.5),
    size:(45,35,25),
    faces:{
      top:{background-color:documentColors[0]}
      front:{background-color:documentColors[4]}
      right:{background-color:documentColors[1]}
    }
  )
}
```

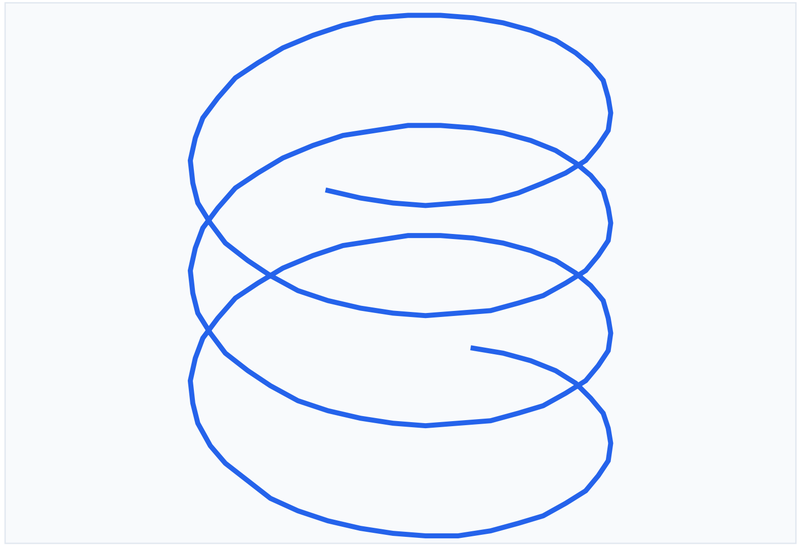


```
scene3d(real-position:(-28,-28,-28), real-size:(56,56,56),
  transform:{scale:0.75}){
    sphere(
        radius:(28,28,28),
        meridians:24,
        parallels:16,
        background-color: documentColors[2]
        contour-color:lighterColor(documentColors[2],0.05)
  )
}
```

3D Primitives

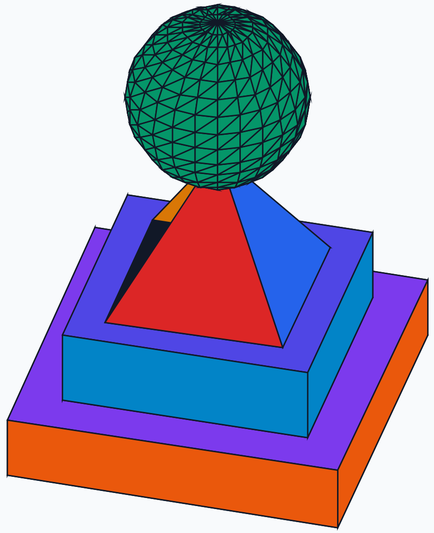


```
scene3d(real-position:(-22.5,-20,-17.5), real-size:(45,40,35), transform:{
  rotate(to:(0,100,0), angle:35)
  scale:0.75
}){
  pyramid(position:(-22.5,-20,-17.5),
    size:(45,40,35),
    faces:{
      front:{background-color:documentColors[4]}
      right:{background-color:documentColors[1]}
      left:{background-color:documentColors[0]}
    }
  )
}
```

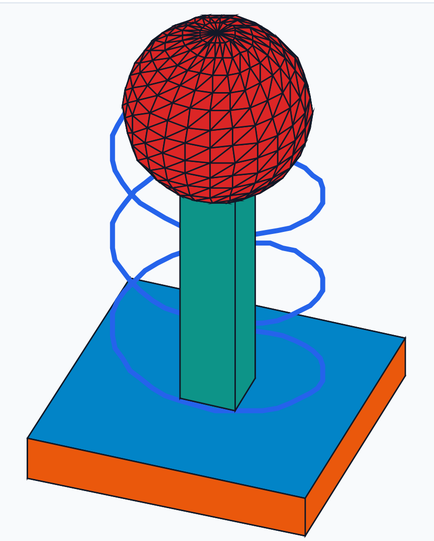


```
scene3d(real-position:(-20,-22,-20), real-size:(40,45,40), transform:{
  rotate(to:(0,100,0), angle:25)
  scale:0.75
}){
  helix(position:(0,-22,0),
    radius:(20,20,20),
    height:45, turns:3.5,
    line-color:documentColors[0],
    stroke:basic(width:2)
  )
}
```

Complex 3D Scenes



```
scene3d(real-position:(0,0,0), real-size:(60,82,60), transform:{
  rotate(to:(0,100,0), angle:30)
  scale:0.65
}){
  // Multi-tier pedestal & monument
  box(size:(60,12,60), position:(0,0,0),
      faces:{top:{background-color:documentColors[1]}})
  box(size:(44,14,44), position:(8,12,8),
      faces:{top:{background-color:documentColors[6]}})
  pyramid(size:(32,28,32), position:(14,26,14),
      faces:{front:{background-color:documentColors[0]},
             right:{background-color:documentColors[3]}})
  sphere(radius:(16,16,16), position:(30,66,30),
      background-color:documentColors[2])
}
```



```
scene3d(real-position:(0,0,0), real-size:(50,82,50), transform:{
  rotate(to:(0,100,0), angle:25)
  scale:0.65
}){
  box(size:(50,8,50), position:(0,0,0),
      background-color:documentColors[10])
  helix(position:(25,8,25),
        radius:(18,18,18),
        height:55, turns:3.0,
        line-color:documentColors[0],
        stroke:basic(width:2))
  box(size:(10,50,10), position:(20,8,20),
      background-color:documentColors[7])
  sphere(radius:(16,16,16), position:(25,66,25),
      background-color:documentColors[4])
}
```

Images


```
image("../images/image.png")
```



```
image("../images/image.jpg")
```



```
image("../images/image.gif")
```



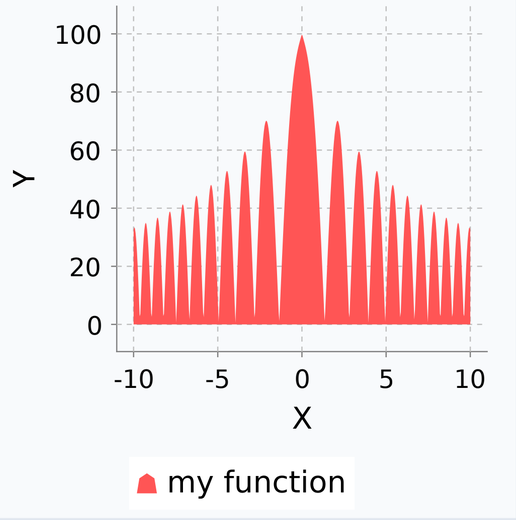
```
image("../images/image.svg")
```



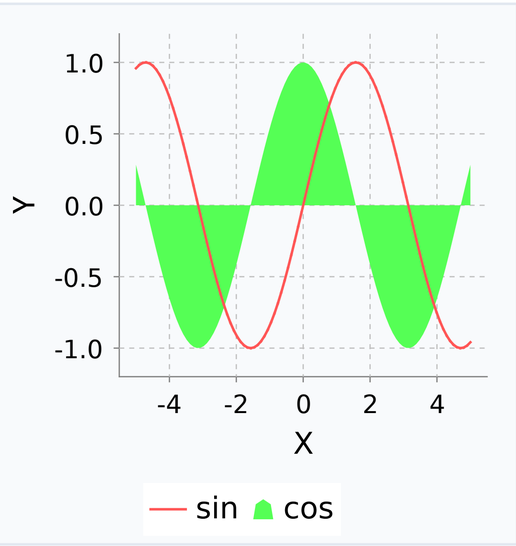
```
image("../images/image.avif")
```

```
image("../images/image.webp")
```

Plot lines

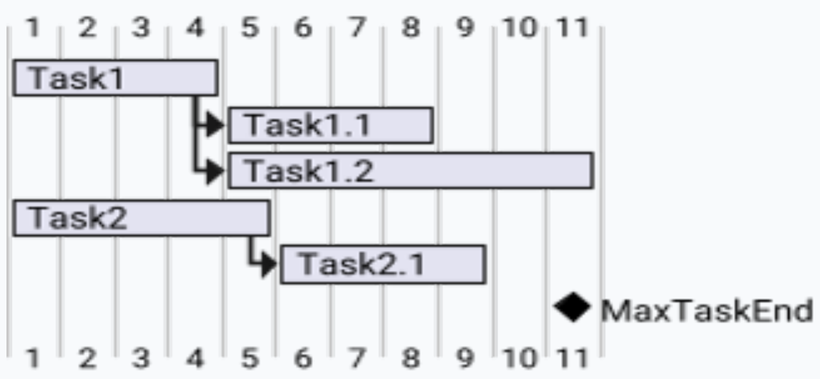



```
plot2d(x:dtimes(-10,10,500)){
    area {
        f(x):"100*abs(cos(pow(abs(x),1.5))/(1+0.2*abs(x)))"
        title:"my function"
    }
}
```




```
plot2d(x:dtimes(-5,5,100)){
    curve{
        f(x):sin(x)
        title:'sin'
    }
    area{
        f(x):cos(x)
        title:'cos'
    }
}
```

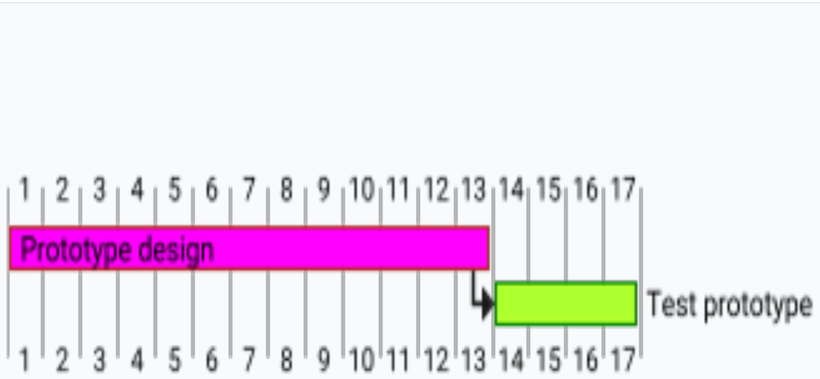
Gantt Diagrams




```
gantt(
    """
    [Task1] requires 4 days
    then [Task1.1] requires 4 days
    [Task1.2] starts at [Task1]'s end and requires 7 days

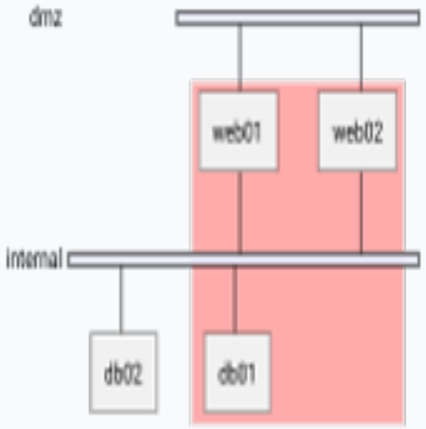
    [Task2] requires 5 days
    then [Task2.1] requires 4 days

    [MaxTaskEnd] happens at [Task1.1]'s end
    [MaxTaskEnd] happens at [Task1.2]'s end
    [MaxTaskEnd] happens at [Task2.1]'s end
    """
)
```



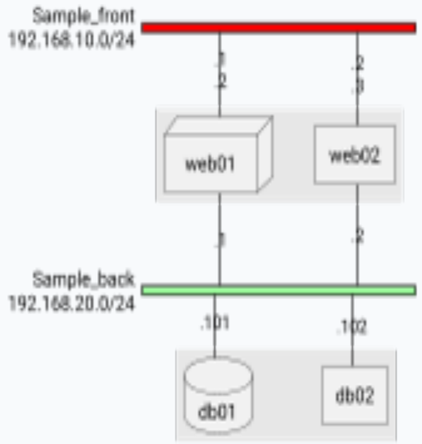

```
gantt(
    """
    [Prototype design] requires 13 days
    [Test prototype] requires 4 days
    [Test prototype] starts at [Prototype design]'s end
    [Prototype design] is colored in Fuchsia/FireBrick
    [Test prototype] is colored in GreenYellow/Green
    """
)
```

Network Diagrams




```
nwdiag(
"""
 // define group outside
 // of network definitions
 group {
   color = "#FFAAAA";
   web01;
   web02;
   db01;
 }

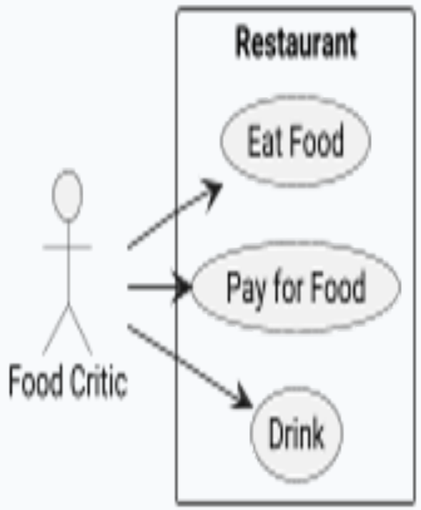
 network dmz {
   web01;
   web02;
 }
 network internal {
   web01;
   web02;
   db01;
   db02;
 }
"""
)
```



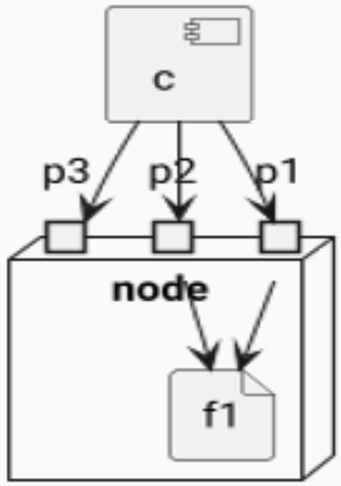

```
nwdiag(
 """
 network Sample_front {
   address = "192.168.10.0/24"
   color = "red"
   // define group
   group web {
     web01 [address = ".1, .2",
      shape = "node"]
     web02 [address = ".2, .3"]
   }
 }
 network Sample_back {
   address = "192.168.20.0/24"
   color = "palegreen"
   web01 [address = ".1"]
   web02 [address = ".2"]
   db01 [address = ".101",
      shape = database ]
   db02 [address = ".102"]

   // define network
   // using defined nodes
   group db {
     db01;
     db02;
   }
 }
 """
)
```

UML Use Case



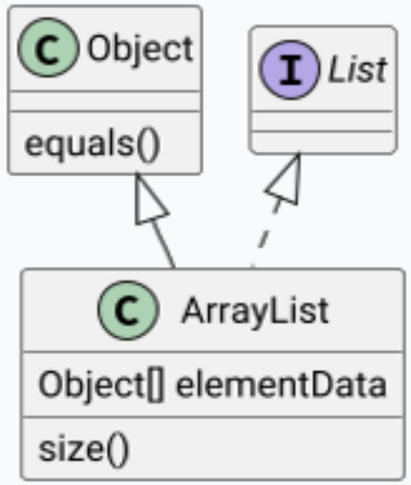

```
uml(
"""
left to right direction
actor "Food Critic" as fc
rectangle Restaurant {
  usecase "Eat Food" as UC1
  usecase "Pay for Food" as UC2
  usecase "Drink" as UC3
}
fc --> UC1
fc --> UC2
fc --> UC3
"""
)
```



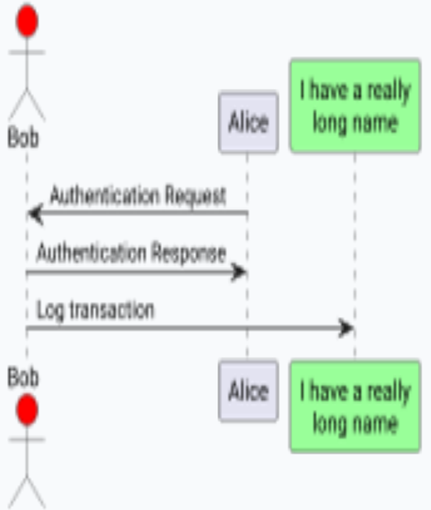

```
uml(
  """
      [c]
      node node {
        port p1
        port p2
        port p3
        file f1
      }

      c --> p1
      c --> p2
      c --> p3
      p1 --> f1
      p2 --> f1
  """
)
```

UML Classes

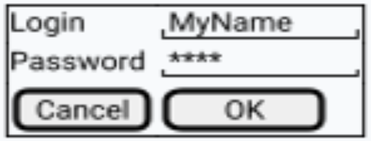



```
uml(
 """
 Object <|-- ArrayList
 List <|.. ArrayList
 interface List
 Object : equals()
 ArrayList : Object[] elementData
 ArrayList : size()
 """
)
```

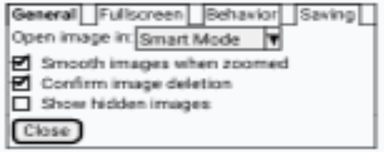



```
uml(
"""
 actor Bob #red
 ' The only difference
 ' between actor
 ' and participant is the drawing
 participant Alice
 participant "I have a really\nlong name" as L #99FF99
 /' You can also declare:
    participant L as "I have a really\nlong name"  #99FF99
  '
 Alice->Bob: Authentication Request
 Bob->Alice: Authentication Response
 Bob->L: Log transaction
"""
)
```

Wireframe Diagrams




```
wireframe(
 """
    Login    | "MyName   "
    Password | "****     "
    [Cancel] | [  OK   ]
 """
)
```




```
wireframe(
 """
     {+
     {/ <b>General | Fullscreen | Behavior | Saving }
     {
     { Open image in: | ^Smart Mode^ }
     [X] Smooth images when zoomed
     [X] Confirm image deletion
     [ ] Show hidden images
     }
     [Close]
     }
 """
)
```

Custom Components (@define)


```
// 1. Define badge with arguments
@define info-badge(label, count: 1, accent: documentColors[0]){
    group(margin: 1, size: (100, 22)){
        rectangle(background-color: accent, round-corner: 4)
        text("${label}: ${count}", color: white,
             font-bold, font-size: 2.2%P, at: center)
    }
}

// 2. Define component wrapping inner 'slot'
@define feature-box(title, accent: documentColors[1]){
    group(margin: 1){
        rectangle(background-color: documentSurface,
                  color: documentLine,
                  draw-contour: true, round-corner: 6)
        rectangle(background-color: accent, position: (0, 0),
                  size: (0.35%P, 100))
        text("Custom Components (@define)", position: (6, 14), origin: left,
             font-bold, font-size: 2.4%P, color: accent)
        group(position: (6, 32), size: (88, 62), origin: top-left){
            slot
        }
    }
}
```

Pending Tasks: 7

Custom Component Box

- Defined via @define name(args){ ... }
- Embeds caller content using slot
- Reusable across slides and layouts

Built-in Template Components

CARD COMPONENT

- Auto-styled container
- Integrated left accent bar
- Embedded kicker support

99.9%

Availability SLA

The best way to predict the future is to invent it.

Alan Kay

Information Callout

Provides contextual tips and info blocks with semantic coloring.

Warning Callout

Highlights cautions or critical notices with warning accent colors.

Panel: Light Canvas

Warm ivory surface for diagrams & light content

Table Extension

- The table extension provides a easy way to create tabular data.
- Supports data, headerData, bodyData, footerData parameters.
- Supports style selectors: table-header(row: n[, col: m]), table-row(row: n | even/odd), table-footer(row: n), table-column(col: n), table-cell(row: r, col: c), table(class-name).
- Sizing: row-weight any row/column/cell se akes the max weight pe nn (per-cell equivalen , 1, 3]). Row indices ar : 1 = first data row); headers/footers via table-header(row: n) / table-footer(row: n).

Simple Table:

Name	Age
Alice	30
Bob	25

Styled Table:

Item	Qty	Price
Apple	5	0.5
Banan	3	0.3
Cherry	10	0.2
Orang	true	1.3
Total		SUM(C1:C4)

Class Usage:

x	y
1	2

Scoreboard Extension

One import, one list.

```
import("scoreboard")

scoreboard(
    title: "Cartoon Duo Challenge",
    scores: [
        {name:"Tom", score:62.5},
        {name:"Rick", score:79.5},
        {name:"Timon", score:74.25}
    ]
)

// or: scoreboard(file: "scores.csv")
```

Also accepts inline CSV, TSON/JSON/YAML files, custom palettes and themes.

Test me

open the following file
<ntexup_gitroot>/documentation/ntexup-doc/02-pages/9999-conclusion/0110-test.ntx
and try to write some things there...

Bye

Questions & Discussion

thevpc

https://github.com/thevpc/ntexup